

COMS 4130 Project Proposal

by Taylor Bauer, Abby Van Der Brink, and Trevor Lis

Project Scope (Taylor)

Application: [infrapy/detection](#) (submodule from [InfraPy](#))

- **Background:** it is an open source python program that was developed by Los Alamos National Lab for low frequency infrasonic soundwave detection from nuclear detonations/large scale explosions and uses an Adaptive Fisher Detector (which is what we will be focusing on)
- **Files:** beamforming_new.py, beamforming.py, spectral.py, visualization.py

Static Analysis Tasks & Tools:

- **Call Graph & CFG:** pycallgraph2, py2cfg for mapping detection flow
- **Dataflow Analysis:** track how detection threshold values are passed & change between functions
- **Abstract Interpretation:** check the ranges of input vals (e.g., signal strength, direction) could cause the detector to make an incorrect decision

Testing Tools:

- **Fuzzing:** pyfuzz, start with example sound wave data (infrapy/examples/data)
- **Coverage:** coverage.py, track line, branch, and path coverage

Initial Design of Query System (Abby)

Storing Queries: The results will be stored in a structured SQLite database with three main tables.

- **Table 1:** Static Analysis Table - will hold call graph edges (caller -> callee relationships), control graph nodes/edges per function, and dataflow dependency pairs
- **Table 2:** Coverage Table - will map each source line and branch to a boolean cover/uncovered status per test run
- **Table 3:** Metadata Table - will store function signatures, the locations of files, and the results of the abstract interpretation

Retrieval Techniques: Queries will be answered in three different ways:

- SQL queries for coverage and dependency lookups
- Graph traversal (BFS/DFS on stored call graph) for reachability questions, such as transitive callers

- Keyword matching for natural language query routing to the correct template

Some Planned Queries:

- *"Is function A a (direct/indirect) caller of function B?"*
 - This will be answered via call graph traversal
- *"What is the branch coverage of function A under fuzzing test suite?"*
 - This will be answered via SQL on the coverage table
- *"Are variable A and B data-dependent?"*
 - This will be answered by SQL lookup on the dataflow dependency table

Timeline and Team Allocation (Trevor - Timeline)

Timeline:

- **Week 7**
 - **Taylor:** install and run InfraPy, explore InfraPy/detection module
 - **Abby:** Draft database schema for SQLite query storage
 - **Trevor:** Review the Adaptive Fisher Detector logic and identify variables relevant to abstract interpretation
- **Week 8**
 - **Taylor:** Generate initial call graph using pycallgraph2 and begin CFG extraction using py2cfg
 - **Abby:** Implement initial SQLite database schema
 - **Trevor:** Identify input parameters for range analysis (signal strength, direction, threshold values)
- **Week 9**
 - **Taylor:** Implement dataflow analysis for threshold value propagation
 - **Abby:** Create database ingestion scripts to store analysis results
 - **Trevor:** Develop abstract interpretation rules for input ranges and potentially incorrect detector decisions
- **Week 10**
 - **Team:** Run the first full analysis pipeline
 - **Abby:** Implement coverage tracking with coverage.py
 - **Trevor:** Configure fuzzing environment using pyfuzz with sample soundwave data
- **Week 11**
 - **Taylor:** Export static analysis results to database format
 - **Abby:** Implement both SQL queries for coverage + dependency lookups and graph traversal logic for caller reachability queries
- **Week 12**

- **Taylor:** Export and format static analysis outputs (call graphs, CFGs, dataflow) so they can be ingested into the database and assist Abby with verifying correctness
- **Abby:** Lead integration of all data into the SQLite query database and ensure schema accommodates static analysis, coverage, and fuzzing results
- **Trevor:** Export and format fuzzing results and abstract interpretation data for database ingestion and help resolve any data compatibility issues
- **Team:** together, verify that queries return correct results using integrated data and perform basic validation tests
- **Week 13**
 - **Team:** Run queries on the detector code to answer example questions such as caller relationships, branch coverage under fuzzing, and data dependencies
 - **Trevor:** Evaluate whether certain input ranges trigger potential incorrect detector behavior
- **Week 14**
 - **Team:** Prepare final report and slides, summarize findings from analysis and testing, and create visualizations of call graphs and coverage results
- **Week 15**
 - **Team:** Present project results, demonstrate query systems with example queries, and discuss limitations and possible future improvements

Team Work Allocation Plan: Review

- **Taylor** - setup & get program running, call graph & CFG, dataflow analysis
- **Abby** - query system design, coverage
- **Trevor** - abstract interpretation, fuzzing